

MELLI Léo
HATTON Victor
COLSON Sasha



Rapport de projet : Pointage mécanique d'antenne



SOMMAIRE

- I. **Introduction**
- II. **Partie mécanique : conceptualisation du Pan & Tilt**
 - A. **Présentation de la partie mécanique du projet**
 - B. **Recherche et conception du bras robotisé**
 - C. **Fabrication et assemblage du bras robotisé**
 - D. **Conclusion et améliorations possibles**
- III. **Partie informatique et électronique : soudure et programmation du moteur**
 - A. **Partie électronique**
 - B. **Partie informatique**
 - C. **Application sur le moteur**
 - D. **Conclusion et points à améliorer**
- IV. **Conclusion générale**

I. Introduction

Nous avons réalisé un projet d'électronique consistant à concevoir et réaliser un système de pointage mécanique d'antenne Wi-Fi. Ce système, basé sur une structure de type Pan & Tilt, permet d'orienter une antenne avec précision grâce à un bras robotisé motorisé.

Le cahier des charges de ce projet s'articule autour de trois objectifs principaux : la conception mécanique, le pilotage électronique et l'asservissement du système. La structure doit être suffisamment robuste pour supporter une antenne de 59 cm tout en restant légère et stable afin de garantir des mouvements fluides sans vibrations parasites. Sur le plan technique, le dispositif utilise des moteurs pas à pas pilotés par des drivers TMC2225 et une carte microcontrôleur STM32, permettant d'atteindre une précision de mouvement indispensable pour un suivi d'antenne fiable. Nous avons dans un premier temps cherché à 3 les premières idées de la conception du bras robotisé. Puis, Léo s'est intéressé à la partie électronique informatique. Sasha et Victor se sont intéressés à la partie mécanique de ce projet.

II. Partie mécanique : conceptualisation du Pan & Tilt

A. Présentation de la partie mécanique du projet

Dans le cadre de notre projet d'électronique, nous avons conçu un système permettant à une antenne Wi-Fi d'être en mouvement grâce à un bras robotisé motorisé.

La partie mécanique du projet avait pour objectif principal de concevoir une structure capable de supporter l'antenne tout en permettant des mouvements précis et fluides. Cette structure devait également être compatible avec les moteurs pas à pas utilisés pour le pilotage du bras.

Le travail réalisé dans cette partie du projet a consisté à :

- rechercher un modèle de bras robotisé adapté ;
- sélectionner les différentes pièces mécaniques nécessaires ;
- concevoir et imprimer certaines pièces en 3D ;
- assembler l'ensemble du bras robotisé ;
- tester la stabilité et corriger les problèmes rencontrés.

L'objectif était d'obtenir une structure suffisamment solide pour maintenir correctement l'antenne, tout en restant légère afin de ne pas surcharger les moteurs.

B. Recherche et conception du bras robotisé

La première étape du projet a été l'étude de différents modèles de bras robotiques existants afin de trouver une structure adaptée à notre besoin. Plusieurs critères ont été pris en compte lors de cette recherche :

- la stabilité ;
- la mobilité ;
- la facilité d'assemblage ;
- la compatibilité avec les moteurs pas à pas ;
- le poids total de la structure.

Comme l'antenne devait rester orientée avec précision, il était important d'éviter les vibrations et les mouvements parasites. Nous avons donc choisi un modèle comportant plusieurs articulations, permettant une bonne amplitude de mouvement tout en conservant une structure compacte. Voici le bras robotisé que nous avons retenu : <https://www.printables.com/model/22610-pan-tilt-wnema17-steppers>

Une fois le modèle sélectionné, nous avons identifié les différentes pièces nécessaires à la réalisation du bras :

- supports mécaniques ;
- axes de rotation ;
- fixations ;
- adaptateurs pour moteurs ;
- éléments de maintien de l'antenne.

Nous avons également vérifié quelles pièces étaient déjà disponibles à l'ENSEA afin de limiter les commandes inutiles et réduire le coût du projet. Voici la liste de nos composants commandés :

Fournisseur				Envoyez un seul fichier par mail			
Pos	I	P	Art	Désignation	Qté commande	UA	Date Livraison
10	Z			barre métallique 8mm 300mm 786-6015	1	PC	16.01.2026
20	Z			Insert M3 4mm 5.7mm 204-0616	1	PC	16.01.2026
30	Z			Insert M4 5.6mm 8.2mm 204-0617	1	PC	16.01.2026
40	Z			Roulement aiguilles 8x21x4mm	4	PC	16.01.2026
50	Z			Vis tête hexagonale 25 mm 190-276	1	PC	16.01.2026
60	Z			Vis 6 pans M3 8mm 281-388	1	PC	16.01.2026
70	Z			Vis 6 pans M3 8mm 187-1213	1	PC	16.01.2026
80	Z			Vis 6 pans M3 12mm 281-007	1	PC	16.01.2026
90	Z			Vis 6 pans M3 20mm 304-4429	1	PC	16.01.2026
100	Z			Vis 6 pans M3 30mm 293-331	1	PC	16.01.2026
110	Z			Vis 6 pans M4 16mm 187-1263	1	PC	16.01.2026
120	Z			Rondelle M3 560-338	1	PC	16.01.2026
130	Z			Rondelle M4 525-925	1	PC	16.01.2026
140	Z			Ecrou hexagonal 1/4 245-0710	1	PC	16.01.2026
150	Z			Ecrou hexagonal M3 189-563	1	PC	16.01.2026
160	Z			Ecrou hexagonal M4 189-579	1	PC	16.01.2026
170	Z			Roulement aiguilles 50x70x5 273-1950	2	PC	16.01.2026
180	Z			Roulement billes 8x22x7 619-0036	6	PC	16.01.2026
190	Z				1	PC	
							EUR

Cette phase de conception a permis de préparer l'assemblage du bras et d'anticiper les contraintes mécaniques liées aux mouvements.

C. Fabrication et assemblage du bras robotisé

Après la phase de conception, nous avons commencé la fabrication des différentes pièces du bras robotisé. Certaines pièces étant spécifiques à notre projet, nous avons utilisé l'impression 3D afin de créer des éléments sur mesure.

L'impression 3D a permis :

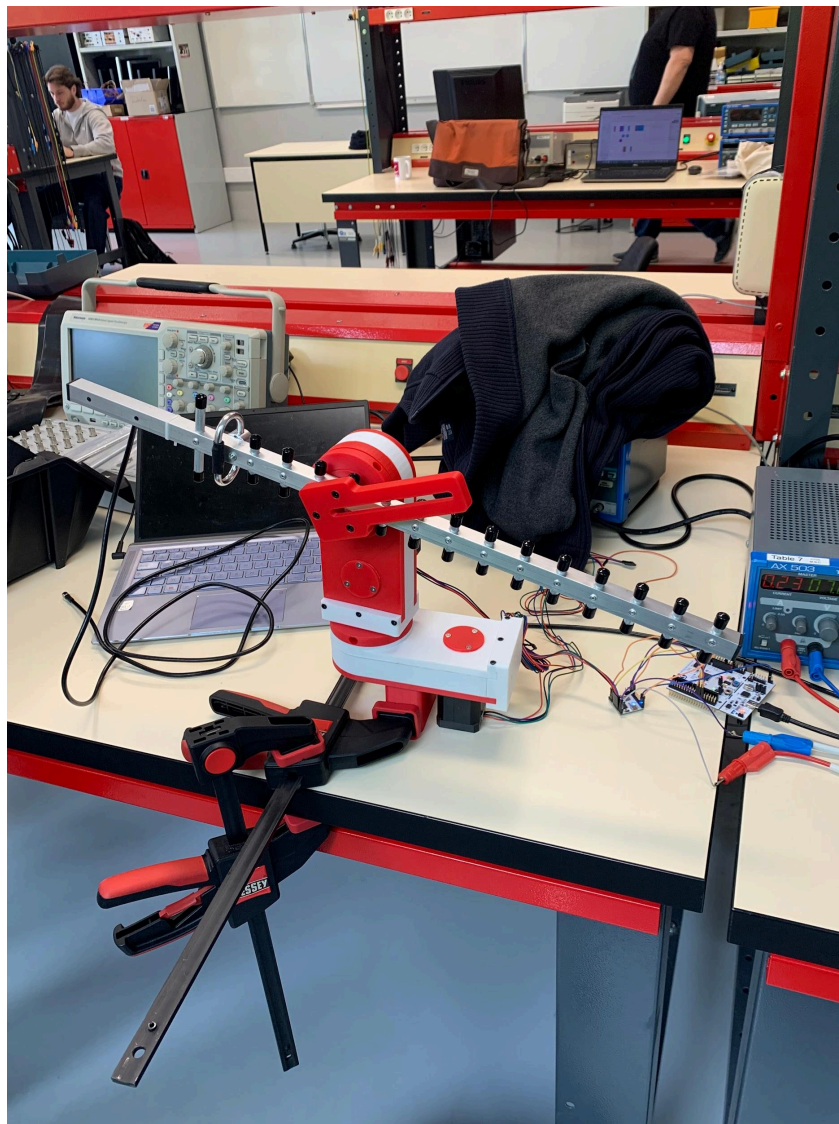
- de fabriquer rapidement des pièces adaptées ;
- de modifier facilement certaines dimensions ;
- de tester plusieurs versions de pièces ;

- de réduire les coûts de fabrication.

Les pièces imprimées servaient notamment à :

- relier les moteurs au bras ;
- maintenir certaines articulations ;
- fixer l'antenne Wi-Fi ;
- adapter certaines dimensions incompatibles avec les pièces standards.

Une fois les pièces imprimées, nous avons procédé à l'assemblage complet du bras robotisé. Cette étape demandait beaucoup de précision afin de garantir un bon alignement des articulations et un fonctionnement fluide des mouvements.



Montage final

Durant cette phase, plusieurs difficultés ont été rencontrées :

- certaines pièces étaient trop fragiles ;
- certaines dimensions n'étaient pas assez précises ;
- du jeu apparaissait dans les articulations ;

Pour résoudre ces problèmes, plusieurs solutions ont été mises en place :

- réimpression de certaines pièces avec des dimensions modifiées ;
- ajustement des fixations ;
- optimisation de la répartition du poids.

Ces différentes modifications ont permis d'améliorer la stabilité et la fluidité des mouvements du bras robotisé.

D. Conclusion et améliorations possibles

La partie mécanique a joué un rôle essentiel dans le fonctionnement global du projet. Elle a permis de créer une base physique stable capable de supporter l'antenne Wi-Fi.

Ce travail nous a permis d'acquérir plusieurs compétences, notamment :

- la recherche et l'adaptation d'un modèle mécanique existant ;
- l'utilisation de l'impression 3D ;
- l'assemblage de systèmes mécaniques ;
- la résolution de problèmes techniques au cours du développement.

Même si le projet fonctionne correctement, plusieurs améliorations pourraient être envisagées pour optimiser la partie mécanique :

- utiliser des matériaux plus résistants pour certaines pièces ;
- réduire davantage le poids du bras ;
- améliorer la précision des articulations ;
- concevoir un support d'antenne plus compact ;
- améliorer l'esthétique générale du bras robotisé.

Ces améliorations permettraient d'obtenir un système plus robuste, plus précis et plus performant pour un futur développement du projet.

III. Partie informatique et électronique: soudure et programmation du moteur

Cette partie se compose de deux parties hétérogènes :

- une partie électronique : choix des composants et soudures
- une partie informatique : programmation et mise en pratique sur le Pan & Tilt

A. Partie électronique

1) Composants

- Contrairement à d'autres groupes de projet d'électronique, nous ne concevons pas de PCB : en effet, seule la **STM** fournie par l'école suffira. Elle permet de commander et d'alimenter.
- Nous avons choisi des **moteurs pas à pas** : ils sont à la fois faciles à commander (car nous avons précédemment fait des TPs avec ces moteurs), mais surtout le modèle du Pan & Tilt repose sur ces moteurs. Il fallait donc respecter le modèle choisi
- Avec les deux moteurs, nous avons choisi de prendre deux **drivers TMC2225** : nous avons opté pour ce choix car ils ont été fournis par notre encadrant. Cela était au final un choix optimal pour comprendre facilement et rapidement comment faire tourner un moteur.
- Nous avons évidemment des **fils** à notre disposition, à la fois des fils d'alimentation, mais aussi des fils femelle-femelle pour pouvoir brancher les ports de la STM aux drivers.
- Une **alimentation de laboratoire** est, enfin, indispensable pour pouvoir faire tourner le moteur : un ordinateur seul ne suffit pas.

Remarque : Nous avons songé un moment à brancher une Raspberry Pi sur notre STM à la place de l'ordinateur : toutefois, par manque de temps et pour éviter de trop se compliquer la tâche, nous avons délaissé cette idée, qui aurait pu très bien fonctionner.

2) Soudure

Nous avons soudé plusieurs composants :

- Les drivers : Soudure des ports à Arès, avec le matériel à disposition
- Pour pouvoir passer de l'alimentation aux ports de la nucléo, nous avons soudé un fil d'alimentation de laboratoire avec un fil femelle-femelle dénudé d'un côté. Nous n'avons pas à disposition des pinces crocodiles, d'où ce choix.

Voici à quoi ressemble notre assemblage au final, avec un driver branché :

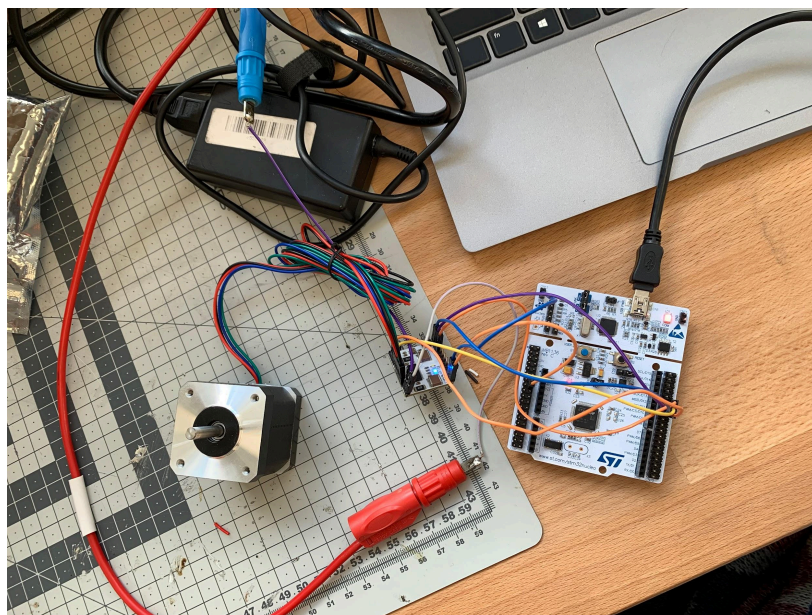


Photo du schéma électrique, avec un driver branché

B. Partie informatique

Cette partie est consacrée à la programmation : nous avons codé en C, à partir du logiciel STM32CubeIDE.

1) Compréhension du driver

Avant toute ligne de code, la compréhension de chaque port du driver était essentielle ; nous avons donc consacré pas mal de temps à comprendre chaque port utile pour notre moteur. En voici un résumé :

- **Port ENN** : fonctionne comme un interrupteur ON/OFF pour le driver. Il possède deux modes : LOW pour alimenter le moteur, et HIGH pour couper le courant.
- **Port DIR** : abréviation de “direction”, il oriente le sens de rotation du moteur.
- **Ports MS1, MS2** : abréviations de “Micro-Stepping”, ils permettent de configurer le nombre de micropas par pas complet.
- **Port STEP** : il est utile car il permet d’envoyer des impulsions au moteur, et donc de le faire tourner (pas à pas dans notre cas)
- **Port GND** : port pour relier la masse.
- **Port Vcc_IO** : port permettant de relier le driver au port 3,3V de la nucléo.
- **Port +Vm** : port permettant de relier le driver à l’alimentation.

D’après la Datasheet de ce driver, on y trouve le schéma suivant :

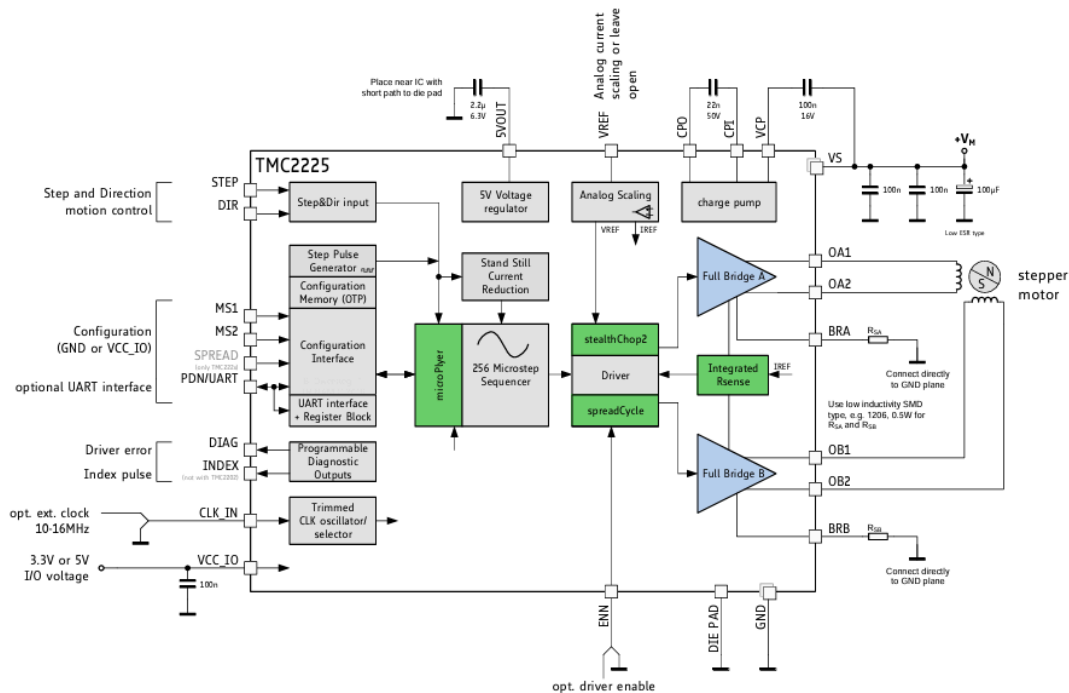


Figure 1.1 TMC2225 basic application block diagram

Alors, pour brancher notre moteur sur un driver, il faut considérer les **ports A1, A2, B1 et B2**. Il faut 4 ports car dans un moteur pas à pas classique, il y a deux électro aimants (bobines), pour pouvoir arrêter le moteur dans une configuration particulière, mais aussi pour pouvoir inverser le sens de rotation du moteur à notre guise.

Il vient alors nos choix de ports sur le nucléo :

DRIVER 1		DRIVER 2	
Nom	Broche GPIO	Nom	Broche GPIO
ENN	PA12	ENN	PB5
DIR	PA7	DIR	PB4
MS1, MS2	PA11, PA12	MS1, MS2	PB1, PB2
STEP	PA6 (TIM3_CH1)	STEP	PB6 (TIM4_CH1)

Ces choix s'inspirent d'un TP que nous avons réalisé en microcontrôleur au premier semestre.

2) Programmation : préliminaires

Vient alors les prémices de notre programmation : comme toute programmation sur STM32CubeIDE, nous allons créer des fichiers `utils.c`, `utils.h`. Nous allons aussi faire un `setup()` dans le `main.c`, pour éviter de travailler directement dans le `main.c`. Toutefois, nous avons été contraints à quand même insérer des fonctions et variables dans le `main`, notamment pour modifier la fonction **HAL_TIM_PeriodElapsedCallback**, mais aussi pour ajouter la fonction **HAL_UART_RxCpltCallback**.

Remarque essentielle : les screens de nos programmes seront pour un driver uniquement (le driver 1), car les programmes sont les mêmes pour les deux drivers, aux noms des broches près.

3) Utils.h

```

8 #ifndef INC_UTILS_H_
9 #define INC_UTILS_H_
10
11 #include "stm32l4xx_hal.h"
12
13 #define SPR 1600.0f
14 #define STEPS_PER_DEGREE (SPR / 360.0f)
15
16
17 void setup(void);
18 void loop(void);
19 void Moteur_AllerA_Angle(float angle);
20 void Moteur_MiseAJour(void);
21

```

Screen du fichier utils.h

Dans ce fichier on y trouve les appels aux fonctions de utils.c (donc sa partie est située après celle-ci).

La ligne **#define SPR (400.0f)** définit le nombre de pas par tour de l'axe final. Cette valeur est le produit des caractéristiques physiques du moteur, de la configuration du driver et du rapport de réduction mécanique. Ici, la valeur de **800 (car $400 = 800/2$)** indique que le driver est configuré en **1/4 de pas** (200×4), ce qui permet un mouvement fluide et silencieux. On multiplie cette valeur par **2** pour intégrer le rapport de réduction de nos poulies (**36 dents / 18 dents**). On obtient donc un total de **400** pas pour un tour complet du bras.

La ligne **#define STEPS_PER_DEGREE (SPR / 360.0f)** établit le facteur de conversion entre l'unité humaine (le degré) et l'unité machine (le pas). On a donc ici environ **1,11 pas par degré**, après calculs.

Remarque: on utilise ici des nombres flottants, pour la précision de notre moteur.

4) Utils.c

```

12 #include "utils.h"
13
14 extern TIM_HandleTypeDef htim3;
15
16 volatile int32_t current_pos = 0;
17 volatile int32_t target_pos = 0;
18
19 void setup(void) {
20     //activation driver
21     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, GPIO_PIN_RESET);
22     HAL_Delay(10);
23
24     //démarrage compteur
25     HAL_TIM_Base_Start_IT(&htim3);
26
27     HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, GPIO_PIN_RESET);
28
29
30     //Moteur_AllerA_Angle(720.0f);
31 }

```

première partie du fichier utils.c

Puisque le moteur tourne lorsqu'on est en mode LOW, alors il faut se mettre en **RESET**. Cela va permettre d'alimenter les bobines du moteur. De plus, puisqu'on s'appuie sur le **Timer 3** pour faire tourner le moteur, il faut bien l'activer: on l'active en mode **Interruption**, pour que le moteur tourne de manière fluide, malgré les autres instructions du programme.

```

33 void Moteur_AllerA_Angle(float angle) {
34     //On définit des limites
35     float angle_securise = angle;
36
37     float angle_min = -180.0f;
38     float angle_max = 180.0f;
39
40
41
42     if (angle_securise > angle_max) {
43         angle_securise = angle_max;
44     }
45     if (angle_securise < angle_min) {
46         angle_securise = angle_min;
47     }
48
49     //On met à jour l'angle
50     target_pos = (int32_t)(angle_securise * STEPS_PER_DEGREE);
51 }

```

Seconde partie du fichier utils.c

Il faut bien une fonction permettant de faire tourner le moteur à sa guise : la fonction **Moteur_AllerA_Angle** nous le permet. Elle permet de mettre à jour l'angle lorsqu'on lui donne une valeur en angle. Elle définit aussi des extremums, ce qui permet à l'antenne de ne pas toucher le sol (car elle est assez longue).

```
53 void Moteur_MiseAJour(void) {
54     // Si on est arrivé à destination, on ne fait rien
55     if (current_pos == target_pos) {
56         return;
57     }
58
59     //direction
60     if (target_pos > current_pos) {
61         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_SET);
62         current_pos++;
63     } else {
64         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_RESET);
65         current_pos--;
66     }
67
68     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET);
69     for(volatile int i=0; i<50; i++);
70     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET);
71
72     // LED témoin
73     HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
74 }
```

Dernière partie du fichier utils.c

Enfin, la fonction **Moteur_MiseAJour**, exécutée périodiquement par une interruption Timer, assure le **pilotage** en générant les impulsions physiques nécessaires pour atteindre cette cible pas après pas. Cette fonction permet d'ajuster le programme **Moteur_AllerA_Angle**, car comme nous le verrons par la suite, il y a des imprécisions lors de l'application à notre bras robotisé. Cette fonction est néanmoins à retravailler, car elle n'applique pas vraiment ce que l'on souhaite.

5) Main.c

Enfin, on initialise dans ce dossier .c des variables globales qui permettent de donner des instructions sur le moteur :

```
50 /* USER CODE BEGIN PV */
51 uint8_t car_recu; // La touche du clavier
52 float angle_actuel = 0.0f; // La mémoire de la position
53
```

On écrit ensuite la fonction `setup()` et la fonction permettant de lancer la première écoute du clavier (HUART2):

```
107  setup(); // Démarre le driver et le Timer (dans utils.c)
108
109  // On lance la première écoute du clavier
110  HAL_UART_Receive_IT(&huart2, &car_recu, 1);
```

Aussi, on écrit la fonction `HAL_UART_RxCpltCallback` : elle permet d'associer la touche "d" pour tourner de 10° dans le sens horaire, et "q" pour tourner de 10° dans le sens antihoraire:

```
178 void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
179     if (huart->Instance == USART2) {
180
181         //On traite la touche
182         if (car_recu == 'd') {
183             angle_actuel += 10.0f;
184         }
185         else if (car_recu == 'q') {
186             angle_actuel -= 10.0f;
187         }
188
189         //On donne l'ordre au moteur
190         Moteur_AllerA_Angle(angle_actuel);
191
192         //On renvoie la touche au PC pour voir ce qu'on fait
193         HAL_UART_Transmit(&huart2, &car_recu, 1, 10);
194
195         //On réécoute pour la prochaine touche
196         HAL_UART_Receive_IT(&huart2, &car_recu, 1);
197     }
198 }
```

Enfin, on ajoute une condition dans la fonction `HAL_TIM_PeriodElapsedCallback`: cette condition permet de vérifier que c'est bien le **Timer 3** qui a fini de compter avant de lancer l'ordre de mouvement.

```
225 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
226 {
227     /* USER CODE BEGIN Callback 0 */
228     if (htim->Instance == TIM3) {
229         Moteur_MiseAJour();
230         //Moteur_MiseAJour2();
231     }
```

Nous avons aussi, évidemment, modifié l'ARR pour que les moteurs tournent suffisamment rapidement, sans être brusques.

C. Application sur le moteur

Voici le rendu final de notre moteur :

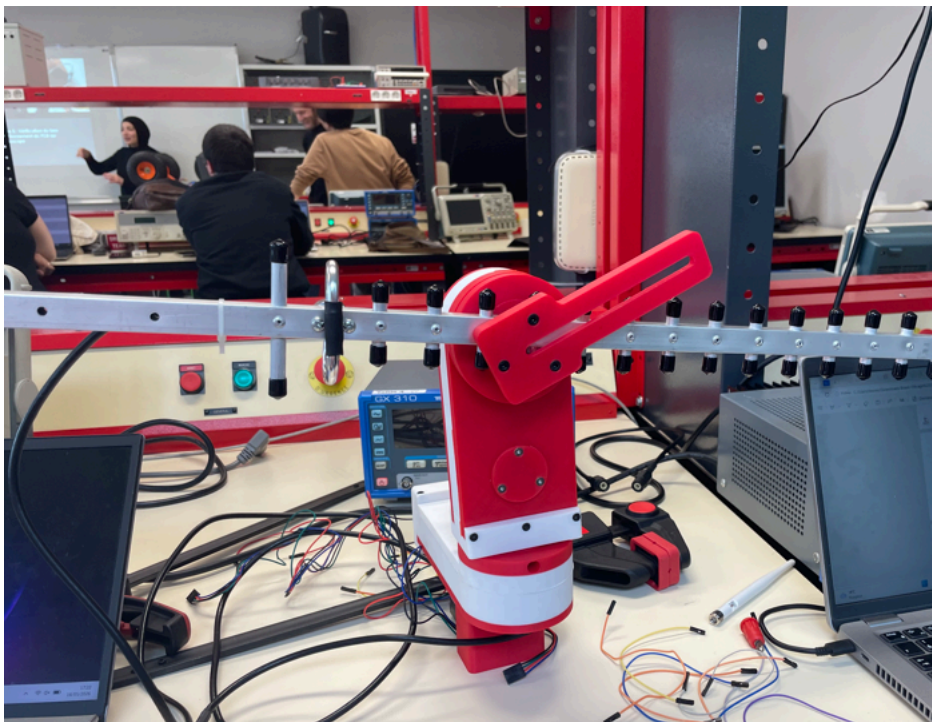


Les programmes fonctionnent bien : il n'y a pas de problèmes dessus. En revanche, nous avons rencontré des difficultés par rapport au moteur sur notre installation directement : le moteur du bas ne tourne vraiment pas correctement : l'installation doit être trop lourde par rapport à la puissance du moteur, ou la courroie n'est pas très efficace, car elle a été imprimée. Le moteur du haut fonctionne mieux : néanmoins cela n'est pas très précis : lorsque l'on veut tourner aux maximums imposés (tourner au maximum de 180°), nous ne sommes pas exactement à 180° .



Exemple avec une vis (en noir) : on remarque effectivement une certaine imprécision.

Nous avons aussi regardé le champ électromagnétique du moteur : il fallait déterminer la configuration où l'antenne renvoie le moins bien son champ par rapport à la clé wifi : en voici cette configuration, qui est à environ 69% :



Description	: TP-Link Wireless USB Adapter
Signal	τá: 69%

Ci-dessus : données trouvées dans le terminal, en branchant une clé Wifi à l'antenne.

D. Conclusion et points à améliorer

Cette partie a été très instructive pour nous : elle nous a permis de mettre en application les connaissances que nous avons vues au cours de l'année en informatique (USART, broches du GPIO, compréhension du driver, moteur).

Il y a des points en informatique et électronique à améliorer dans notre projet:

- Mettre des capteurs magnétiques sur les axes pour augmenter la précision des angles; car, comme vu précédemment, il y des imprécisions.
- Améliorer les courroies, qui sont assez fragiles, ou augmenter le courant sur les drivers, ou améliorer le SPR, qui n'est peut-être pas assez précis (car certes, on a un rapport de 2 avec les poulies, mais cela n'est peut-être pas assez précis de mettre 1600).
- Mieux ordonner nos branchements, car le fait d'avoir chaque fil séparément était assez contraignant à la longue.
- Brancher nos deux drivers en parallèle pour avoir qu'une seule alimentation.

IV. Conclusion générale

En conclusion, ce projet de système de pointage mécanique d'antenne Wi-Fi nous a permis de valider avec succès le pilotage électronique via une carte STM32 et des drivers TMC2225, ainsi que la conception d'une structure Pan & Tilt fonctionnelle. Bien que nous ayons rencontré des difficultés notables concernant la précision mécanique, la fragilité des courroies imprimées et le poids de l'ensemble, le dispositif répond aux objectifs de mobilité. Un axe d'amélioration est par exemple, l'utilisation de matériaux plus robustes, l'ajout de capteurs de position magnétiques pour corriger les imprécisions angulaires et l'optimisation de l'alimentation en parallèle constituent des évolutions majeures pour rendre ce bras robotisé pleinement performant.